



中山大學
SUN YAT-SEN UNIVERSITY

From C to C++

中山大学计算机学院



主讲人：郑贵锋



联系方式：zhenggf@mail.sysu.edu.cn

目录

CONTENTS

01

string 类

02

string 的操作

03

使用C标准库

04

const & **constexpr**

05

auto, bool 关键字



string 类 - 初识类

- 什么是类（Class）？
 - 类是现实世界或抽象世界中**概念**，是关于同类事物定义的描述。
 - 向面向对象语言中，声明猫类Cat 相关的属性如 color 和行为 catch() 等。则只要是 Cat 类别的对象如 tom，就有color属性和catch行为。
 - 计算机语言中**类是模板**，它是关于成员对象**数据成员**和**函数成员**的描述
 - 因此，Class 是一种数据类型
- 在标准库中有很多类，通过以下类可体验与C语言的不同
 - <string>库中的 `std::string` 类
 - 输入/输出库中的 `std::ifstream`, `std::ofstream` 类



string 类 - vs C 风格字符串

- C++ 提供了以下两种类型的字符串表示形式：
 - C 风格字符串
 - C++ 的 string 类 类型
- 为什么要引入 string 类型的对象？
 - C 风格字符串是使用 null 字符 '\0' 终止的一维字符数组。因而
 - 字符串定义时，必须事先知道保留多大空间存储字符串
 - 在字符串操作，如 strcat，时要防止目标字符串有足够的空间
 - 使用 malloc 动态空间处理字符串，对一般程序员来说太复杂，非常容易出现BUG



string 类 - 创建 string 对象

```
/* string-create-objects*/
#include<iostream>
#include<string>
using namespace std;

int main() {
    string s1;
    string s2 = "c++";
    string s3 = s2;
    string s4 (5, 's');

    cout << s3 << endl;
    //请自己输出 s1,s2,s4
    return 0;
}
```

- 为什么是创建对象，不是声明变量？
 - 对象通常需要复杂的初始化过程，常需要参数
 - s1 未使用初始化参数，即默认初始化为空字符串
 - **对象类型 对象 = 参数1**，表示用参数1初始化对象。这里 s2 使用C字符串初始化；s3 用 s2 初始化（又称拷贝复制）。
 - **对象类型 对象 (参数列表)** 适用有参数的对象初始化。这里 s4 初始化为 5 个 s；
 - string s2 = "c++" **不是**赋值运算，它等价于 string s2("C++") **是**初始化



string 类 - string 对象的运算

string类实现了以下运算符的重载

- 联接运算：
 - `operator+`, 连接两个字符串或者一个字符串和一个字符
- 比较运算：
 - `operator==, !=, <, <=, >, >=, <=>` (*c++20起*), 以字典序比较两个字符串
- 下标运算：
 - `operator[]`, 访问指定字符, 返回指定字符引用, 即可作为左值
- 赋值运算：
 - `operator=, +=`, 右操作数可以是字符串, 字符, C字符串, 字符数组



string 类 - string 对象运算示例

```
/* string-ops.cpp */
#include<iostream>
#include<string>
using namespace std;

int main() {
    string s1="hello ",s2,s3;
    cin >> s2;
    s3 = s1 + s2;
    cout << s3 << endl;

    s1 += {'s','y','s','u'}; //没\0
    cout << s1 << endl;
    //请自己编程测试其他运算
    return 0;
}
```

```
#include<iostream>
#include<string>
using namespace std;

int main() {
    string s1 = "abc";
    string s2 = "def";
    if (s1 > s2) cout << s1+s2;
    else cout << s2[1];

    return 0;
}
```

输出结果为: [填空1]

作答



string 对象操作

string类实现了许多 string 对象的操作（成员函数）

- 基本操作：length, c_str
- 查找：find, rfind
- 添加，插入，删除：append, insert, erase, clear
- 提取子字符串：substr
- 比较：compare
- 替换：replace
- 复制与交换：copy, swap
- 数值转换（C++11）：stoi, stod

以上仅列举了目前需要掌握的字符串操作，在学完 STL 后，可参考：std::basic_string - cppreference.com



string 操作 - 基本操作

- 基本操作：
 - `length()` 返回类型 `size_t` 的字符串的长度
 - `c_str()` 返回类型 `const char *` 的 C 风格字符串。以 `null` 结束

```
/* str-method-basic.cpp */  
#include<iostream>  
#include<string>  
using namespace std;  
  
int main() {  
    string s = "sysu-computer";  
    int len = s.length();  
    const char* cs = s.c_str();  
    cout << cs << " len is "  
        << len << endl;  
    return 0;  
}
```



string 操作 - 查找、替换

- 查找、替换：
 - `find(string|char*|char s [,int pos=0])` 寻找首个等于给定字符序列的子串。搜索始于 `pos`，返回类型 `size_t` 的位置。`rfind` 从右边开始搜索
 - `replace(int pos, int count, string|char*|char s)` 替换指定范围内容

```
/* str-method-find.cpp */  
#include<iostream>  
#include<string>  
using namespace std;  
  
int main() {  
    string s = "sysu-computer";  
    int pos = s.find("computer");  
    s.replace(pos,8,"software");  
    cout << s << endl;  
    return 0;  
}
```



string 操作 - 函数重载

在C语言中，一个函数只能定义一个函数名以及相关形参。

C++ 则允许多个函数拥有相同的名字，只要它们的参数列表不同就可以，这就是函数的重载 (*Function Overloading*)。借助重载，一个函数名可以有多种用途。

以 string 对象的 find 函数为例：

- `s.find(str)` 在 `s` 中查找子串 `str`
- `s.find("hello", 3)` 在 `s` 中从第4个位置开始查找子串 "hello"
- `s.find('H')` 在 `s` 中查找字符 'H'

这种使用一个函数名，实现了多种查找功能，提升了程序的可读性。让编译器根据函数参数不同，选择合适的函数版本，这种让编译“聪敏”一些，是C++扩展C语言的重要手段之一。



string 操作 - 添加, 插入, 删除

- 添加, 插入, 删除:
 - `append(string)` 后附 string str
 - `append(int count, char ch)` 后附 count 个字符 ch
 - `insert(int index, int count, char ch)` 在 index 位置插入 count 个 ch
 - `insert(int index, string|char* s)` 在 index 位置插入 s
 - `erase(int index, int count)` 删除从 index 开始的 count 个字符
 - `clear()` 清空

```
/* str-method-insert.cpp */  
#include<iostream>  
#include<string>  
using namespace std;  
  
int main() {  
    string s = "ysu";  
    s.insert(0,1,'S').append(1,'-').append("Xcomputer");  
    s.erase(5,1);  
    cout << s << endl;  
    return 0;  
}
```



string 操作 - 子串、比较、拷贝、交换

- 子串、比较、拷贝、交换：
 - substr(int pos, int count) 返回子串对象
 - compare(string|char* str) 与str比较, 返回 1, 0, -1之一
 - compare(int pos, int count, string|char* str) 子串与str比较, 返回 1, 0, -1之一
 - copy(string|char* dest, int pos, int count) 将子串复制到目标对象
 - swap(string other) 与 other 交换内容

```
/* str-method-substr.cpp */
#include<iostream>
#include<string>
using namespace std;

int main() {
    string s = "Sysu-Computer";
    cout << s.substr(0,4) << " ";
    if (s.compare(5,8,"Computer")==0) {
        cout << "OK!" << endl;
    }
    return 0;
}
```

```
/* str-method-insert.cpp */  
#include<iostream>  
#include<string>  
using namespace std;  
  
int main() {  
    string s = "abc";  
    s.insert(0,1,'A').append("fde");  
    s.erase(4,1);  
    cout<<s.substr(4,2);  
    return 0;  
}
```

输出结果为: [填空1]

作答



使用 C 标准库

C++ 标准库提供了可以在标准 C++ 中使用的各类设施。也包括

- C 库设施的 C++ 标头(header, 头文件), 即在 std 空间中的 c 标准库
- C 标准库, 即不仅可以用 g++ 编译 c 风格的 cpp 程序, 也可以使用 C 库函数

提供了 C 库设施的 C++ 标头			
<cassert>	<ctype.h>	<csignal>	<csignal.h>
<cctype>	<cmath>	<csdlib>	<csdlib.h>
<cerrno>	<csetjmp>	<csdlib>	<csdlib.h>
<cfloat>	<csignal>	<csdlib>	<csdlib.h>
<climits>			
C++20 中移除的标头			
<ciso646>			
C++11 中添加的标头			
<cfenv>	<csignal>	<csdlib>	<csdlib.h>
C++11 中添加, C++17 中弃用, C++20 中移除的标头			
<ccomplex>	<cstdalign>	<cstdbool>	<ctgmath>

头文件名命: c+C标头文件名

C 标头			
<assert.h>	<limits.h>	<stdarg.h>	<string.h>
<ctype.h>	<locale.h>	<stddef.h>	<time.h>
<errno.h>	<math.h>	<stdio.h>	<wchar.h>
<float.h>	<setjmp.h>	<stdlib.h>	<wctype.h>
<iso646.h>	<signal.h>		
C++11 中添加的标头			
<complex.h>	<inttypes.h>	<stdbool.h>	<tgmath.h>
<fenv.h>	<stdalign.h>	<stdint.h>	<uchar.h>
C++23 中添加的标头			
<stdatomic.h>			



案例研究 - 回文测试

回文的一些例子:

- Did Anna say as Anna Did?
- No lemons, no melon.
- Was it a rat I saw?
- A man, a plan, a canal: Panama.
- Able was I ere I saw Elba.
- 问题求解
 - 去除非字母单词, 并变为小写, 在<ctype.h>中有许多需要的函数
 - 判断 Palindrome 算法 ...



案例研究 - 回文测试部分实现 (1)

```
/* c-plindrome.cpp */  
#include<iostream>  
#include<string>  
#include<ctype>  
#include<cassert>  
  
using namespace std;  
  
namespace sysu {  
    string low_alpha(const string& str);  
    bool test_palindrome(const string& str);  
}
```

声明要点:

- 正确使用 C 库设施, 这里是 ctype 和 assert 库
- 在 app 模块的 namespace 中声明全局变量、对象、函数和类等标识符



案例研究 - 回文测试部分实现 (2)

```
/* c-plindrome.cpp */
string sysu::low_alpha(const string& str) {
    string s;
    int len = str.length();
    for (int i=0; i<len ;i++) {
        if (isalpha(str[i])) {
            s.append(1,tolower(str[i]));
        }
    }
    return s;
}
```

```
/* c-plindrome.cpp */
int main() {
    using sysu::low_alpha;
    string s;
    getline(cin,s);
    s = low_alpha(s);
    //请课上或课后完成回文测试函数
    assert(s[0] == s[s.length()-1]);
    cout << s;
    return 0;
}
```

要点:

- 使用限定名 `sysu::` 实现命名空间下的函数, 注意 `isalpha` 等 `c` 函数在 `std` 命名空间声明
- 使用 `getline` 读入字符串到 `s`



const 和 constexpr (c++11)

- C语言中 `const` 关键字限定类型变量不可修改。在右示例代码中, `dis_1()` 和 `dis_2()` 函数中都包含一个 `const int x`, 但 `dis_1()` 函数中的 `x` 无法完成初始化 `array` 容器的任务, 而 `dis_2()` 函数中的 `x` 却可以。因为 `const` 语义有两个:
 - 限定为不可修改 (只读) 变量
 - 限定作为常量或字面量
- C++11 标准新添加的关键字 `constexpr`, 声明**编译时**可以对函数或变量求值。即
 - 限定为常量表达式
 - 限定为编译时可优化执行的函数

```
#include <iostream>
#include <array>
using namespace std;

void dis_1(const int x){
    // 错误, x是只读的变量
    array <int,x> myarr{1,2,3,4,5};
    cout << myarr[1] << endl;
}

void dis_2(){
    const int x = 5;
    array <int,x> myarr{1,2,3,4,5};
    cout << myarr[1] << endl;
}

int main()
{
    dis_1(5);
    dis_2();
}
```



const 和 constexpr (C++11)

这是因为，`dis_1()` 函数中的 “`const int x`” 只是想强调 `x` 是一个只读的变量，其本质仍为变量，无法用来初始化 `array` 容器；而 `dis_2()` 函数中的 “`const int x`”，表明 `x` 是一个只读变量的同时，`x` 还是一个值为 5 的常量，所以可以用来初始化 `array` 容器。

C++ 11标准中，为了解决 `const` 关键字的双重语义问题，保留了 `const` 表示“只读”的语义，而将“常量”的语义划分给了新添加的 `constexpr` 关键字。因此 C++11 标准中，建议将 `const` 和 `constexpr` 的功能区分开，即凡是表达“只读”语义的场景都使用 `const`，表达“常量”语义的场景都使用 `constexpr`。

```
#include <iostream>
#include <array>
using namespace std;

void dis_1(const int x){
    // 错误, x是只读的变量
    array <int,x> myarr{1,2,3,4,5};
    cout << myarr[1] << endl;
}

void dis_2(){
    const int x = 5;
    array <int,x> myarr{1,2,3,4,5};
    cout << myarr[1] << endl;
}

int main()
{
    dis_1(5);
    dis_2();
}
```



auto 关键字 - 占位类型说明符

auto的原理就是根据初始化的表达式，在编译期间根据表达式类型自动推导出对象的类型。auto的作用就是为了简化变量初始化，如果这个变量有初始化表达式，就可以用auto代替类型声明。例如

```
auto x1 = 5; 等价于 int x1 = 5;
```

注意点：

1. 用auto声明的变量必须初始化
2. 函数和模板参数不能被声明为auto
3. 因为auto是一个**类型占位符**，并不是一个类型，因此不能用于类型转换或其他一些操作，如sizeof和typeid
4. 定义在一个auto序列的变量必须始终推导成同一类型

```
auto x1 = 5, x2 = 5.0, x3 = 'r'; //这样写是不合法的，因为各个变量类型不同
```



auto

比如下面的代码：

```
vector<int> a;  
vector<int>::iterator it = a.begin();
```

定义it对应的变量类型时，名字有点长。

我们在初始化it的时候可以这样写：

```
auto it = a.begin();
```

而在遍历迭代器的时候我们也可以用auto简化代码：

```
int main(){  
    vector<int>v;  
    v.push_back(1);  
    v.push_back(2);  
    v.push_back(3);  
    for(auto i : v){  
        cout<<i<<" ";  
    }  
    cout<<endl;  
    return 0;  
}
```



bool 类型

bool 类型在 C++ 中是基础类型（在c中不是！）：

- bool - 是存放两个值 true 或 false 之一的类型。
- sizeof(bool) 的值由实现定义，而且不一定是 1。
- bool, true, false 是关键字。



小结

Form C to C++, 有许多变化:

- 支持面向对象的编程
 - 如, 输入/输出对象 `cin`, `cout`, `String` 类
- 扩展了语言新特性
 - 如, 命名空间, 引用, 运算符、函数重载等
- 提供了C标准库基础设施
 - 如, 提供了两种方式使用 C 标准库, 方便使用 C 资源
- 对C语言已有内容完善和补充
 - 如, `const`, `auto`, `bool` 等内容



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组